



Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Αναφορά 2ου εργαστηρίου
“Ψηφιακά Συστήματα VLSI”

ΟΝΟΜΑΤΕΠΩΝΥΜΑ	Χαλβατζόγλου Ιωάννα (ΑΜ: el21906) Αϊφαντή Ελισάβετ (ΑΜ: el22821)
ΟΜΑΔΑ	Ομάδα 13
ΔΙΔΑΣΚΟΝΤΕΣ	Κ.Πεκμεστζή Δ.Σούντρης Η. Παπαλάμπρου

Άσκηση 1:

❖ *VHDL code*

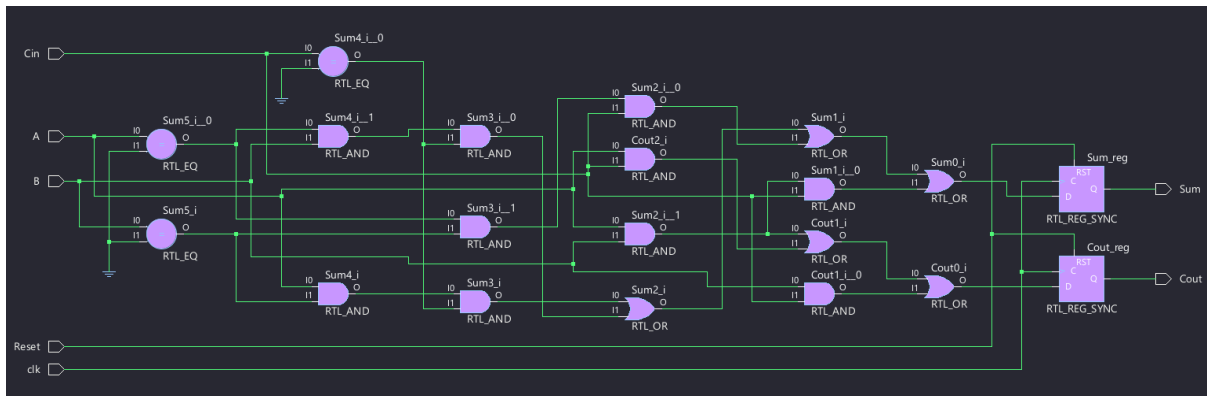
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder is
    port (
        A, B, Cin, Reset, clk : in std_logic;
        Sum, Cout: out std_logic );
end full_adder;

architecture Behavioral of full_adder is
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if Reset = '1' then
                Sum <= '0'; Cout <= '0';
            else
                if (A = '1' and B = '0' and Cin = '0') or
                    (A = '0' and B = '1' and Cin = '0') or
                    (A = '0' and B = '0' and Cin = '1') or
                    (A = '1' and B = '1' and Cin = '1') then
                    Sum <= '1';
                else
                    Sum <= '0';
                end if;

                if (A = '1' and B = '1') or
                    (A = '1' and Cin = '1') or
                    (B = '1' and Cin = '1') then
                    Cout <= '1';
                else
                    Cout <= '0';
                end if;
            end if;
        end if;
    end process;
end Behavioral;
```

❖ RTL Schematic



Παρατηρούμε πως η behavioral περιγραφή δίνει ένα ισοδύναμο κύκλωμα σε αυτό που υλοποιεί τον πλήρη αθροιστή με πύλες xor, αναλύοντας τη σύνθετη αυτή πύλη με or και and πύλες.

❖ Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder_tb is
end full_adder_tb;

architecture Behavioral of full_adder_tb is
    signal A, B, Cin, Reset : std_logic := '0';
    signal clk : std_logic;
    signal Sum, Cout : std_logic;

    constant clk_period : time := 5 ns;

begin
    UUT: entity work.full_adder
        port map (
            A => A,
            B => B,
            Cin => Cin,
            Reset => Reset,
            clk => clk,
            Sum => Sum,
            Cout => Cout
        );

    process
    begin
        clk <= '0';
        wait for clk_period/2;
```

```

    clk <= '1';
    wait for clk_period/2;
end process;

process
begin
    Reset <= '1'; wait for clk_period;
    Reset <= '0';
    A <= '0'; wait for 80ns; A <= '1'; wait for 80ns;
end process;

process
begin
    wait for clk_period;
    B <= '0'; wait for 40ns; B <= '1'; wait for 40ns;
end process;

process
begin
    wait for clk_period;
    Cin <= '0'; wait for 20ns; Cin <= '1'; wait for 20ns;
end process;

```

end Behavioral;

Το testbench επιβεβαιώνει τη λειτουργία του σύγχρονου αθροιστή.

❖ *Timing Report*

Name	Slack ^{^1}	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock
↳ Path 1	∞	2	2	1	Cout_reg/C	Cout	4.063	3.264	0.800	∞	
↳ Path 2	∞	2	2	1	Sum_reg/C	Sum	4.063	3.264	0.800	∞	
↳ Path 3	∞	2	3	2	B	Sum_reg/D	1.920	1.120	0.800	∞	input port clock
↳ Path 4	∞	2	3	2	B	Cout_reg/D	1.894	1.094	0.800	∞	input port clock
↳ Path 5	∞	1	2	2	Reset	Cout_reg/R	1.770	0.970	0.800	∞	input port clock
↳ Path 6	∞	1	2	2	Reset	Sum_reg/R	1.770	0.970	0.800	∞	input port clock

Το κρίσιμο μονοπάτι από την είσοδο B στην έξοδο Sum έχει χρονική καθυστέρηση 5.983ns.

Άσκηση 2:

❖ *VHDL code*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity pipelined_adder4 is
port(
    clk : in std_logic;
    a : in std_logic_vector(3 downto 0);
    b : in std_logic_vector(3 downto 0);
    cin : in std_logic;
    sum : out std_logic_vector(3 downto 0);
    cout : out std_logic
);
end pipelined_adder4;

architecture Behavioral of pipelined_adder4 is

    signal c1,c2,c3,c4 : std_logic;

    signal s0,s1,s2,s3 : std_logic;

    signal a_reg1,a_reg2,a_reg3 : std_logic_vector(3 downto 0);
    signal b_reg1,b_reg2,b_reg3 : std_logic_vector(3 downto 0);

    signal c_reg1,c_reg2,c_reg3 : std_logic;

    signal s0_reg1, s0_reg2, s0_reg3 : std_logic;
    signal s1_reg2, s1_reg3 : std_logic;
    signal s2_reg3 : std_logic;

begin

    -- stage 1
    s0 <= a(0) xor b(0) xor cin;
    c1 <= (a(0) and b(0)) or (cin and (a(0) xor b(0)));
```

```

-- stage 2
s1 <= a_reg1(1) xor b_reg1(1) xor c_reg1;
c2 <= (a_reg1(1) and b_reg1(1)) or (c_reg1 and (a_reg1(1) xor b_reg1(1)));

-- stage 3
s2 <= a_reg2(2) xor b_reg2(2) xor c_reg2;
c3 <= (a_reg2(2) and b_reg2(2)) or (c_reg2 and (a_reg2(2) xor b_reg2(2)));

-- stage 4
s3 <= a_reg3(3) xor b_reg3(3) xor c_reg3;
c4 <= (a_reg3(3) and b_reg3(3)) or (c_reg3 and (a_reg3(3) xor b_reg3(3)));

```

```

process(clk)
begin
    if rising_edge(clk) then

```

```

        --stage 1
        a_reg1 <= a;
        b_reg1 <= b;
        c_reg1 <= c1;
        s0_reg1 <= s0;

```

```

        --stage 2
        a_reg2 <= a_reg1;
        b_reg2 <= b_reg1;
        c_reg2 <= c2;
        s0_reg2 <= s0_reg1;
        s1_reg2 <= s1;

```

```

        --stage 3
        a_reg3 <= a_reg2;
        b_reg3 <= b_reg2;
        c_reg3 <= c3;
        s0_reg3 <= s0_reg2;
        s1_reg3 <= s1_reg2;
        s2_reg3 <= s2;

```

```

        end if;
    end process;

```

```

sum <= s3 & s2_reg3 & s1_reg3 & s0_reg3;
cout <= c4;

```

```

end Behavioral;

```

❖ *Testbench*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity pipelined_adder4_tb is
-- Port ();
end pipelined_adder4_tb;

architecture testbench of pipelined_adder4_tb is

    signal clk : std_logic := '0';
    signal a,b : std_logic_vector(3 downto 0);
    signal cin : std_logic := '0';
    signal sum : std_logic_vector(3 downto 0);
    signal cout : std_logic;

    constant clock_delay : time := 10 ns;

begin
    dut: entity work.pipelined_adder4(Behavioral)
    port map(
        clk => clk,
        a => a,
        b => b,
        cin => cin,
        sum => sum,
        cout => cout
    );

    stimulus: process
    begin

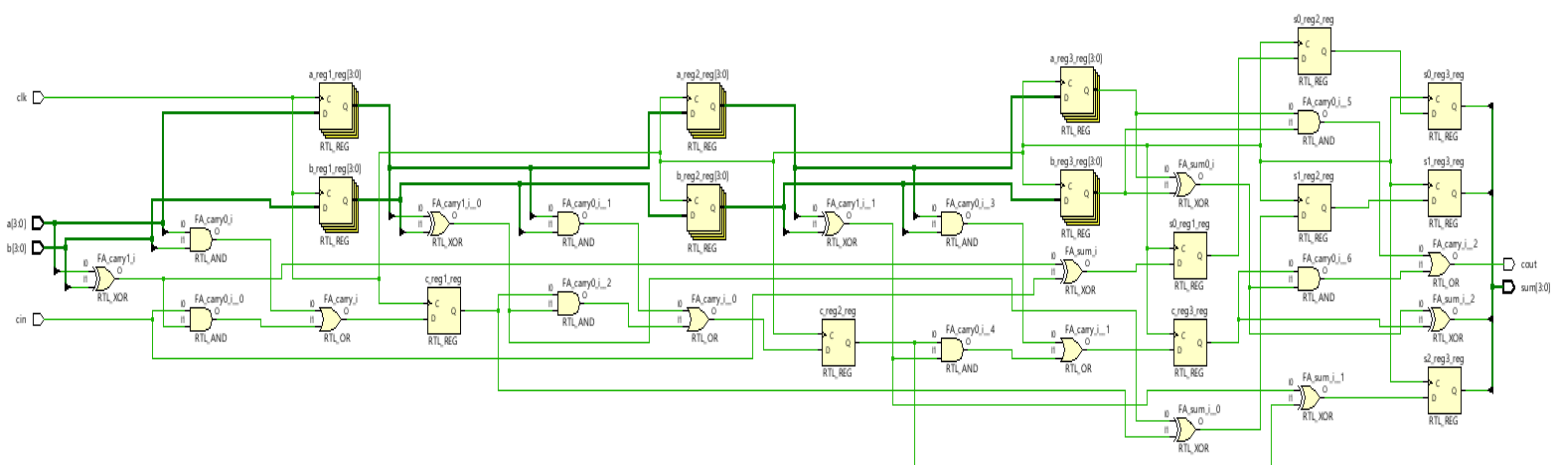
        a <= (others => '0');
        b <= (others => '0');
        cin <= '0';
        wait for clock_delay;

        loop
```

end process;

```
gen_clock: process
begin
clk <='0';
wait for (clock_delay/2);
clk <='1';
wait for (clock_delay/2);
end process;
```

```
end testbench;
```

❖ *RTL schematic*

Οι καταχωρητές εδώ έχουν προστεθεί για αποθήκευση προηγούμενης τιμής (ανάδραση), συγχρονισμό με το ρολόι και παράλληλη επεξεργασία δεδομένων.

❖ *Timing report - Critical path*

Name	Slack ^{^1}	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Logic %	Net %	Requirement	Source
Unconstrained Paths (1)													
(none) (10)													
↳ Path 11	∞	3	2	2	b_reg3_reg[3]/C	cout	5.872	3.432	2.440	58.4	41.6	∞	
↳ Path 12	∞	3	2	2	b_reg3_reg[3]/C	sum[3]	5.609	3.179	2.430	56.7	43.3	∞	
↳ Path 13	∞	2	1	1	s2_reg3_reg/C	sum[2]	5.109	3.244	1.865	63.5	36.5	∞	
↳ Path 14	∞	2	1	1	s0_reg3_reg/C	sum[0]	5.063	3.256	1.807	64.3	35.7	∞	
↳ Path 15	∞	2	1	1	s1_reg3_reg/C	sum[1]	4.869	3.061	1.807	62.9	37.1	∞	
↳ Path 16	∞	2	2	2	a[0]	s0_reg2_reg_srl2/D	3.061	1.102	1.959	36.0	64.0	∞	input p
↳ Path 17	∞	2	1	2	a[0]	c_reg1_reg/D	2.510	1.074	1.437	42.8	57.2	∞	input p
↳ Path 18	∞	1	1	1	a[3]	a_reg1_reg[3]/D	2.004	0.972	1.032	48.5	51.5	∞	input p
↳ Path 19	∞	1	1	1	a[1]	a_reg1_reg[1]/D	1.853	0.938	0.915	50.6	49.4	∞	input p
↳ Path 20	∞	1	1	1	b[1]	b_reg1_reg[1]/D	1.853	0.956	0.896	51.6	48.4	∞	input p

Αυτά είναι τα αποτελέσματα του timing report από όπου φαίνεται ότι το κρίσιμο μονοπάτι, δηλαδή το μονοπάτι με την μεγαλύτερη καθυστέρηση, είναι το path 11 που αφορά τον καταχωρητή b του 4ου bit έως την έξοδο cout.

Άσκηση 3:

Έγινε χρήση alias mult_cell βασισμένο στον σύγχρονο πλήρη αθροιστή του πρώτου ερωτήματος για τη σχεδίαση του συστολικού 4-bit πολλαπλασιαστή. Βασίζεται στην παραγωγή 16 πλήρων αθροιστών, που αναπαριστούν το grid των partial products A_0B_0 , A_1B_0 , A_2B_0 , A_3B_0 στην πρώτη γραμμή ($i=0$), A_0B_1 , A_1B_1 , A_2B_1 , A_3B_1 στη δεύτερη γραμμή ($i=1$) κ.ό.κ, με j τη στήλη που αναπαριστά το bit του A .

- Κάθε full adder δέχεται το partial product A_jB_i
- Εάν είναι αθροιστής με $i>0$ και $j<3$, δέχεται το άθροισμα του παραπάνω αθροιστή
- Εάν είναι αθροιστής με $i>1$ και $j<3$ δέχεται διαγώνια το carry του άνω δεξιά αθροιστή.

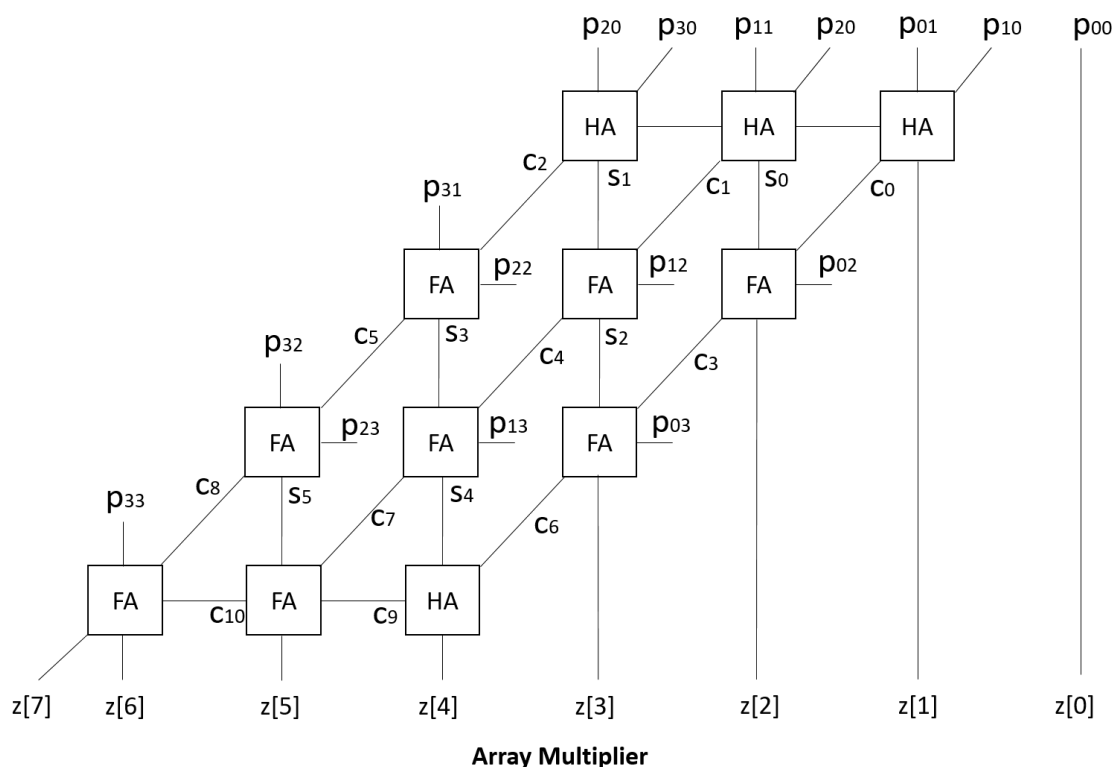
Χρησιμοποιείται signal skewing για την αποθήκευση των input A , B , ώστε ο πολλαπλασιασμός να εφαρμόζεται στα σήματα $A(0)$, $B(0)$ στον πρώτο κύκλο ρολογιού, $A(1)$, $B(1)$ στον δεύτερο κύκλο ρολογιού (όπου πλέον $A(0)$, $B(0)$ αντιστοιχούν στα καινούρια inputs) κ.ό.κ, για τη συστολική λειτουργία του κυκλώματος.

Για το αποτέλεσμα, γίνεται συλλογή των:

- `sum_vect = '0' & sum_grid(15 downto 12) & sum_grid(8) & sum_grid(4) & sum_grid(0);`
- `carry_vec <= carry_grid(15) & carry_grid(14) & carry_grid(13) & carry_grid(12) & "0000";`

Το άθροισμά τους δίνει το αποτέλεσμα του πολλαπλασιασμού των A , B 9 κύκλους μετά.

Σημειώνεται πως έγινε προσπάθεια συλλογής των sum και carry bits με χρήση pipeline registers βάσει του Latency κάθε bit, ωστόσο δεν ήταν επιτυχής. Ο πολλαπλασιαστής δίνει το ορθό αποτέλεσμα κάθε 9 κύκλους.



❖ VHDL CODE

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

-- Implementation of a pipelined systolic 4 bit multiplier using full adders and signal skewing

entity SystolicMult is
  port (
    A, B : in std_logic_vector(3 downto 0);
    rst, clk : in std_logic;
    Mult: out std_logic_vector(7 downto 0)
  );
end SystolicMult;

architecture Behavioral of SystolicMult is
  -- flattened systolic array (i increasing top down, j increasing right to left)
  signal sum_grid, carry_grid, partial_product_grid : std_logic_vector(15 downto 0) := (others => '0');
  signal sum_vec, carry_vec : std_logic_vector(7 downto 0);

  type pipe_array is array (0 to 7) of std_logic_vector(7 downto 0);
  signal a_pipe, b_pipe : pipe_array := (others => (others => '0'));
  signal a_skewed, b_skewed : std_logic_vector(3 downto 0);

  signal valid_bit : std_logic := '0';
  constant LATENCY : integer := 9;
  signal counter : integer := 0;

begin

  process(clk)
  begin
    if rising_edge(clk) then
      for i in 0 to 3 loop
        -- Shift registers for input skewing
        a_pipe(i) <= a_pipe(i)(6 downto 0) & A(i); -- skewed for the first 4 clock cycles, then 4
        cycles remain for latency
        b_pipe(i) <= b_pipe(i)(6 downto 0) & B(i);
      end loop;
    end if;
  end process;

  skew_gen: for i in 0 to 3 generate
    a_skewed(i) <= a_pipe(i)(i);
    b_skewed(i) <= b_pipe(i)(i);
  end generate;
```

```

row_gen: for i in 0 to 3 generate
  col_gen: for j in 0 to 3 generate

    constant idx : integer := i * 4 + j;
    signal prev_sum, prev_carry : std_logic := '0';

    begin

    partial_product_grid(idx) <= A_skewed(j) and B_skewed(i);

    prev_sum_exists: if (i > 0 and j < 3) generate
      prev_sum <= sum_grid(idx - 3);
    end generate;

    prev_carry_exists: if (i > 1 and j < 3) generate
      prev_carry <= carry_grid(idx - 4);
    end generate;

    cell: entity work.mult_cell
    port map (
      Sum_in => prev_sum,
      partial_product => partial_product_grid(idx),
      Cin => prev_carry,
      Reset => rst,
      clk => clk,
      Sum_out => sum_grid(idx), -- propagate sum to row below
      Cout => carry_grid(idx) -- propagates diagonally
    );
  end generate col_gen;
end generate row_gen;

process(clk)
begin
if rising_edge(clk) then
  if rst = '1' then
    valid_bit <= '0';
    counter <= 0;
  else
    -- Tlatency = 8 clock cycles for the first result to appear, after that each result shows every
clock cycle
    if counter < LATENCY-1 then
      counter <= counter + 1;
      valid_bit <= '0';
    else
      valid_bit <= '1';
    end if;
  end if;
end if;
end if;

```

```

end if;
end process;

```

```

sum_vec <= '0' & sum_grid(15 downto 12) & sum_grid(8) & sum_grid(4) & sum_grid(0);
carry_vec <= carry_grid(15) & carry_grid(14) & carry_grid(13) & carry_grid(12) & "0000";

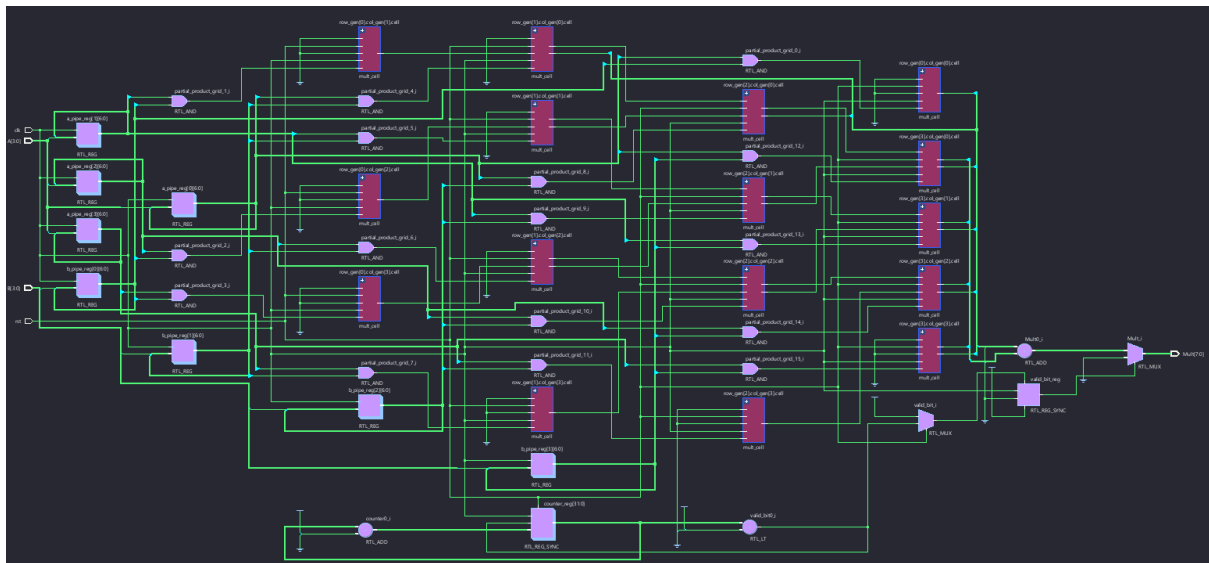
```

```

Mult <= std_logic_vector(unsigned(sum_vec) + unsigned(carry_vec)) when valid_bit = '1' else
(others => '0');
end Behavioral;

```

❖ RTL Schematic



❖ Testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity SystMult_tb is
end SystMult_tb;

```

architecture Behavioral of SystMult_tb is

```

signal A, B : std_logic_vector(3 downto 0) := "0000";
signal rst, clk: std_logic := '0';
signal Mult: std_logic_vector (7 downto 0) := "00000000";

```

```

constant clk_period : time := 5 ns;
signal LATENCY : integer := 9;

```

```

begin

```

UUT: entity work.SystolicMult

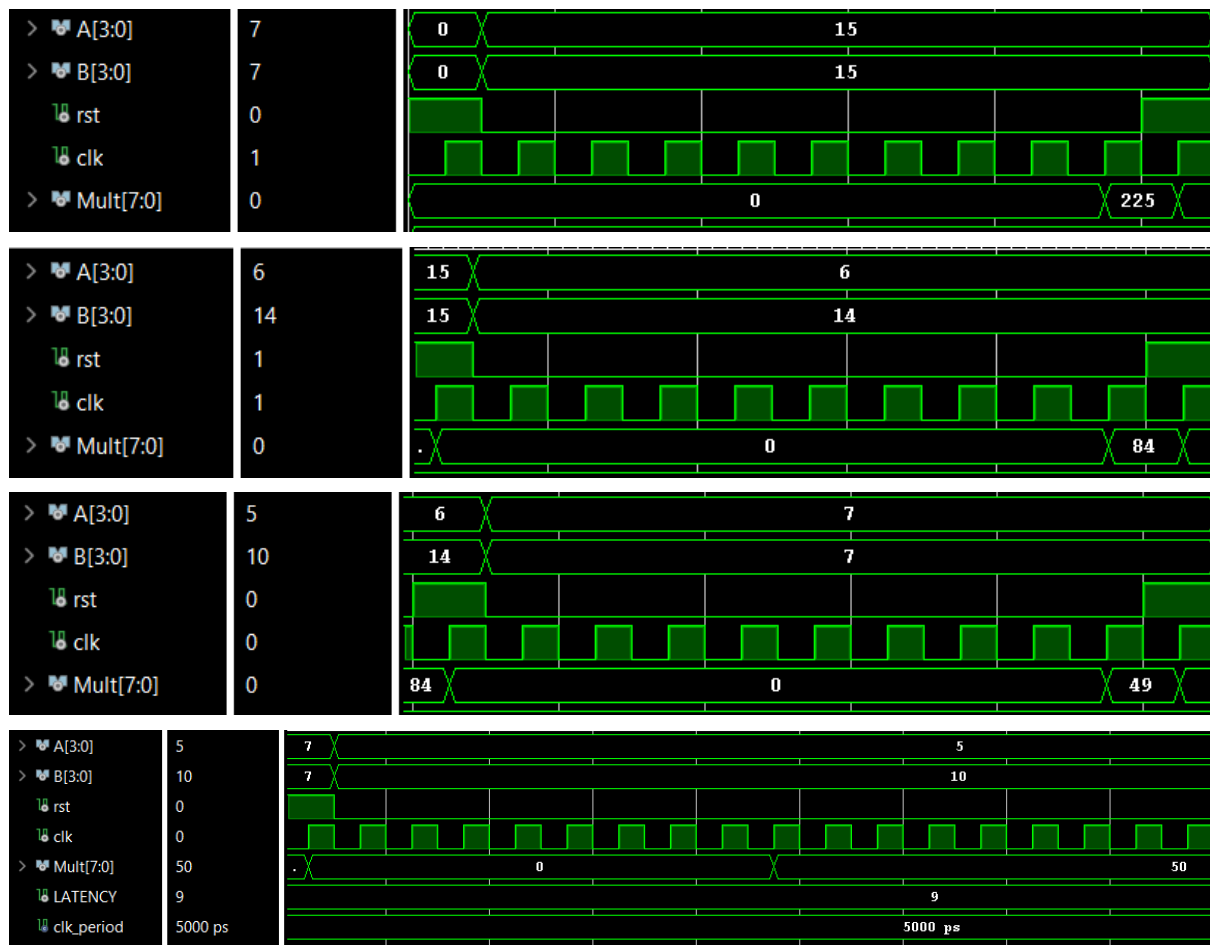
```
port map (  
    A => A,  
    B => B,  
    rst => rst,  
    clk => clk,  
    Mult => Mult );
```

```
process  
begin  
    clk <= '0';  
    wait for clk_period/2;  
    clk <= '1';  
    wait for clk_period/2;  
end process;
```

```
process  
begin  
    rst <= '1';  
    wait for clk_period;  
    rst <= '0';  
  
    A <= "1111"; B <= "1111"; wait for clk_period * LATENCY;  
  
    rst <= '1';  
    wait for clk_period;  
    rst <= '0';  
  
    A <= "0110"; B <= "1110"; wait for clk_period * LATENCY;  
  
    rst <= '1';  
    wait for clk_period;  
    rst <= '0';  
  
    A <= "0111"; B <= "0111"; wait for clk_period * LATENCY;  
  
    rst <= '1';  
    wait for clk_period;  
    rst <= '0';  
  
    A <= "0101"; B <= "1010"; wait for clk_period * LATENCY;  
  
wait;  
end process;
```

end Behavioral;

Με unsigned decimal radix παρατηρούμε πως παράγονται ορθά αποτελέσματα.



❖ Timing Report

Name	Slack	Levels	Routes	Hi...	From	To	Total ... ▼ 1	Logic Delay	Net Delay
↳ Path 1	∞	4	4	2	row_gen[3].col_gen[2].cell/cell/Sum_reg/C	Mult[6]	5.545	3.534	2.012
↳ Path 2	∞	4	4	2	row_gen[3].col_gen[2].cell/cell/Sum_reg/C	Mult[7]	5.545	3.534	2.012
↳ Path 3	∞	3	3	8	valid_bit_reg/C	Mult[5]	5.027	3.384	1.644
↳ Path 4	∞	3	3	8	valid_bit_reg/C	Mult[4]	4.988	3.410	1.579
↳ Path 5	∞	3	3	8	valid_bit_reg/C	Mult[0]	4.691	3.384	1.308
↳ Path 6	∞	3	3	8	valid_bit_reg/C	Mult[1]	4.691	3.384	1.308
↳ Path 7	∞	3	3	8	valid_bit_req/C	Mult[2]	4.691	3.384	1.308

Το κρίσιμο μονοπάτι από το multiplier cell με $i=3$, $j=2$ ($A2 * B3$) από τον καταχωρητή του αθροίσματος στην έξοδο Mult(6) έχει χρονική καθυστέρηση 5.545ns.